

Cahier de Charge: Omnicare

1 Project Overview

Project Name Omnicare

Direct Statement:

The Omnicare platform helps Tunisian families solve the anxiety of medical emergencies by instantly connecting them with verified, trackable doctors for home visits, while providing doctors with a streamlined logistics and monetization tool.

Context / Problem:

In Tunisia, accessing urgent home medical care is highly fragmented, slow, and lacks transparency. Patients rely on outdated phone directories or word-of-mouth, leading to long wait times and uncertainty during emergencies. Furthermore, critical medical history is often fragmented across paper *carnets de santé*, causing delays in accurate diagnosis. On the supply side, doctors lack a centralized, secure system to manage emergency home visits, optimize their travel routes, and guarantee transparent fee collection.

Vision:

To become the standard digital infrastructure for on-demand, real-time healthcare in Tunisia, transforming how families access medical professionals and manage their medical records.

Objectives (SMART)

- **Specific:** Deploy a fully functional dual-sided mobile-responsive web app connecting patients and doctors to the Grand Tunis area.
- **Measurable:** Onboard an initial fleet of 50 verified doctors and process 500 successful home consultations within the first 3 months of launch.

- **Achievable:** Focus Phase 1 strictly on general medicine and high-demand specialties (e.g., Pediatrics) using a static API/mock payment flow before integrating bank gateways.
- **Relevant:** Directly addresses the high demand for reliable, fast home healthcare in urban Tunisian centers.
- **Time-bound:** Deliver the MVP (Minimum Viable Product) ready for beta testing within 4 months.

Target Users / Personas

- **Persona 1: The "Chef de Famille" (Patient Side):** A tech-literate adult (18-45 years old) who manages healthcare logistics not just for themselves, but for their dependents (children and elderly parents). They value speed, reliability, and have medical data centralized.
- **Persona 2: The Mobile Doctor (Provider Side):** A verified general practitioner or specialist looking to optimize their downtime, increase their income through home visits, and utilize digital tools for prescriptions and route planning without the overhead of running a physical clinic 24/7.

Market Positioning

A premium, high-trust, real-time medical logistics platform. Unlike static medical directories or appointment booking sites, OmniCare positions itself as an "Uber for Healthcare," offering live tracking, active bidding, and an integrated digital Medical Passport.

Scope Definition

Included (In-Scope for MVP - Phase 1)

The MVP will focus strictly on the core loop: Requesting an urgent doctor, tracking their arrival, managing the consultation records, and calculating platform fees.

- **Patient & Doctor Authentication:** Phone number/Email login, role selection, and profile creation.

- **Family Management (Chef de Famille):** Creation and management of sub-profiles with individual "Digital Medical Passports" (QR code, blood type, chronic conditions, current medications).
- **7-Step Emergency Request Flow:** Step-by-step wizard capturing symptoms, location (GPS/Manual), and patient details for immediate dispatch.
- **Doctor Dispatch & Bidding System:** Real-time push of urgent requests to nearby doctors, allowing them to accept and input estimated arrival times.
- **Real-Time Live Tracking:** Map interface showing the doctor's movement towards the patient's location.
- **Fast-Closing Consultation Form:** Doctor-side form utilizing quick-select chips for diagnosis, checkboxes for actions taken, and a camera upload feature for paper prescriptions.
- **Patient Review System:** Post-consultation confirmation, emoji-based rating, and tip calculation.
- **Financial "Debt" Engine:** Automated tracking of the 15% platform commission owed by doctors per consultation, displayed in a dedicated revenue dashboard.

Phase 2 (Future Scope - Practice Management Expansion)

These features are part of the platform's vision but are strictly deferred to Phase 2 to ensure a timely Phase 1 launch.

- **Scheduled Doctor Booking:** Allowing patients to browse doctor profiles, view available time slots, and book routine/non-urgent appointments days or weeks in advance.
- **Doctor Consultation Management Dashboard:** A comprehensive SaaS web dashboard for doctors to manage their calendar, view upcoming scheduled appointments, review past patient medical histories, and manage their availability status.
- **Video Consultations:** Remote telemedicine capabilities for psychologists or follow-up visits.
- **Direct Pharmacy Integration:** Sending digital prescriptions directly to the nearest *Pharmacie de Garde*.
- **Automated Insurance Claims:** Integration with CNAM (Caisse Nationale d'Assurance Maladie) or private insurance providers.

Excluded (Strictly Out of Scope for MVP)

The following will **NOT** be developed in the current project phase to actively avoid scope creep and legal/technical bottlenecks:

- **Live Payment Gateway Integration:** The MVP will calculate debts and show checkout UIs, but actual credit card processing (e.g., ClicToPay, Stripe) via external banking APIs is excluded. Payments are assumed to be Cash-on-Delivery (Espèces) between patient and doctor.
- **In-App Video/Voice Calling:** Communication will rely on standard GSM phone calls/SMS by exposing phone numbers, avoiding the heavy infrastructure of WebRTC for the MVP.
- **Ambulance / SAMU Dispatch:** The platform coordinates individual doctors only. Dispatching emergency vehicles involves severe legal and logistical liabilities outside the software's scope.
- **Complex Chat System:** No internal WhatsApp-style messaging system; users will communicate via direct phone calls once matched.

Functional Requirements

This section defines the core capabilities the OmniCare platform must deliver to fulfill its business objectives for the MVP.

3.1 Authentication & Authorization

Feature 3.1.1: Patient Registration & Secure Login

- **Description:** Allows patients to create an account and authenticate securely.
- **Actor:** Patient
- **Input:** Google OAuth token OR Email/Password, followed by Phone Number (+216) and SMS OTP.
- **Output:** Authenticated user session and generated user profile.

- **Business Rules:** * The system must force SMS OTP verification before granting access to the dashboard.
 - Users must complete their profile (Full Name, Age, Default Location) during initial onboarding.

Feature 3.1.2: Doctor Onboarding (Invite-Only)

- **Description:** A closed-loop system ensuring only verified medical professionals access the provider app.
- **Actor:** Doctor, Platform Admin
- **Input:** Admin dashboard entry of Doctor details (Name, Specialty, Phone, CNOM ID).
- **Output:** Provisioned doctor account with generated login credentials.
- **Business Rules:** * Doctors cannot self-register via the public application.
 - Only accounts with the Verified_Doctor role can receive emergency dispatch requests.

3.2 User Profile & Medical Passport (Carnet de Santé Digital)

Feature 3.2.1: Manage Personal Medical Data

- **Description:** Patients maintain a centralized, digital health record.
- **Actor:** Patient
- **Input:** Blood Type (Dropdown), Chronic Conditions (Multi-select tags e.g., Diabète Type 2, Hypertension), Current Medications (Dynamic list: Name, Dosage, Frequency).
- **Output:** Saved Medical Passport data entity.
- **Business Rules:** * Data is fully editable by the patient.
 - The assigned doctor is granted temporary, read-only access to this data only from the moment they accept a request until the consultation is closed.

Feature 3.2.2: Secure Document Management (Vault)

- **Description:** Patients upload and organize medical artifacts.
- **Actor:** Patient
- **Input:** File upload (Image/PDF), Category selection (Analyse de Sang, Radiographie, Ordonnance, Rapport Médical), Date.

- **Output:** Chronological list of accessible medical documents.
- **Business Rules:** * Documents are strictly scoped to the patient and their actively assigned doctor.

Feature 3.2.3: Family Profiles (Chef de Famille)

- **Description:** A primary user manages health records for dependents.
- **Actor:** Patient
- **Input:** Relationship (Baba, Maman, Enfant), Name, Age, and full Medical Passport/Document data.
- **Output:** Navigable sub-profiles linked to the parent account.
- **Business Rules:** * The system applies the exact same medical schema (Passport + Vault) to all family members.
 - Emergency requests can be triggered specifically for a selected family member, passing their specific data to the doctor.

3.3 Emergency Request Flow (The Matching Engine)

Feature 3.3.1: Submit Urgent Request Wizard

- **Description:** A multi-step flow capturing critical triage and location data.
- **Actor:** Patient
- **Input:** * Step 1: Patient Selection (Self or Family Member).
 - Step 2: Symptom details (Interactive Body Map selection, Pain level 1-10, text description).
 - Step 3: Location (GPS coordinates, Street, Apartment).
 - Step 4: Doctor Selection/Bidding view.
 - Step 5: Payment method selection (Cash/Card info).
- **Output:** A structured emergency request payload broadcasted to the system.
- **Business Rules:** * The system must not allow request submission without valid GPS coordinates.

Feature 3.3.2: Doctor Dispatch & Live Tracking

- **Description:** Routing the request to providers and tracking the accepted doctor.
- **Actor:** System, Doctor, Patient
- **Input:** Patient coordinates, Doctor GPS pings.
- **Output:** Match confirmation and live map visualization.

- **Business Rules:** * The system broadcasts the request to verified, online doctors within a specific geographic radius.
 - Once a doctor accepts, the system locks the request (1-to-1 match).
 - The system must stream the doctor's live GPS coordinates to the patient's map interface until the status changes to Arrived.

3.4 Consultation Closure & Financial Engine

Feature 3.4.1: Fast-Report Closing (Clôture de Consultation)

- **Description:** A high-speed form for doctors to document the visit before leaving the patient.
- **Actor:** Doctor
- **Input:** Diagnosis (Pill/Tag selection), Vital Signs (Blood pressure, sugar, temp), Treatments Performed (Checkboxes), Camera capture of handwritten prescription.
- **Output:** Consultation status updated to Completed.
- **Business Rules:** * The form relies on tap/select inputs to eliminate manual typing for the doctor.
 - The photo of the prescription taken by the doctor is automatically saved into the Patient's Document Vault under the "Ordonnances" category.

Feature 3.4.2: Platform Debt Engine

- **Description:** Automated calculation of platform commissions.
- **Actor:** System
- **Input:** Final consultation fee entered/confirmed at closing.
- **Output:** Updated financial ledger for the doctor.
- **Business Rules:** * The system automatically calculates 15% of the total consultation fee.
 - This 15% is added to the doctor's Platform Debt balance.
 - Actual fee collection is handled via Cash (Espèces) between patient and doctor.

Feature 3.4.3: Patient Review & Confirmation

- **Description:** Closes the loop on the patient side.
- **Actor:** Patient

- **Input:** Completion confirmation, Emoji-based rating (😞 , 😐 , 😊), optional tip amount.
- **Output:** Updated doctor rating and finalized transaction record.
- **Business Rules:** * If the patient fails to confirm within a set timeout (e.g., 60 minutes) after the doctor marks it complete, the system auto-confirms the closure.

Non-Functional Requirements

This section defines the system's quality attributes, performance goals, and security standards required to ensure a reliable and legally compliant medical platform.

4.1 Performance & Responsiveness

- **Real-Time Latency:** The live map tracking (doctor's GPS ping to patient's screen) must have a latency of **under 200 milliseconds** to ensure smooth UI movement without jumping.
- **Emergency Load Time:** The core "Request Doctor" screen must be fully interactive within **2 seconds** of app launch on a standard 4G mobile network.
- **API Response Time:** All standard database queries (e.g., loading the Medical Passport) must return within **300 milliseconds** (95th percentile).

4.2 Scalability

- **Concurrent Users:** The system architecture must support a minimum of **1,000 concurrent active connections** (WebSocket/Real-time) for the Phase 1 MVP (Grand Tunis area) without performance degradation.
- **High-Frequency Writes:** The location engine must be capable of absorbing GPS pings every 3 seconds from at least 100 active doctors simultaneously without locking the primary relational database.

4.3 Security & Privacy (Medical Grade)

- **Data Encryption:** * **In Transit:** All data exchanged between the client and server must be encrypted using **TLS 1.2+** (HTTPS/WSS).

- **At Rest:** Sensitive medical records (Blood type, conditions, medications) and uploaded artifacts (Prescription images) must be encrypted at rest using **AES-256**.
- **Access Control (RBAC):** Strict Role-Based Access Control must be enforced via JWT (JSON Web Tokens). A doctor's token must automatically lose access rights to a patient's Medical Passport the moment the consultation status changes to Completed or Cancelled.
- **Authentication Security:** OTPs (One-Time Passwords) sent via SMS must expire within **5 minutes** and allow a maximum of 3 failed attempts before enforcing a 15-minute cooldown.

4.4 Availability & Reliability

- **Uptime Target:** Given the urgent medical nature of the application, the core backend services must guarantee **99.9% uptime**.
- **Offline Tolerance / Edge Cases:** * If a doctor loses 4G connection while filling out the "Fast-Report Closing" form, the mobile client must cache the data locally and automatically sync to the server once the connection is restored.
 - If a transaction fails mid-process (e.g., calculating the 15% platform debt), the database must use **ACID-compliant rollbacks** to prevent corrupted financial states.

4.5 Compliance & Legal

- **Tunisian Data Protection:** The system must comply with **Loi Organique N° 2004-63** (Protection des données à caractère personnel).
 - Explicit user consent must be captured before storing health data or sharing location.
 - Patients must have a "Right to Delete" their medical vault artifacts permanently.
- **Audit Logging:** All sensitive actions—specifically when a doctor accesses a patient's medical passport or an Admin modifies a doctor's verification status—must be recorded in an immutable audit log containing the User ID, Action, and Timestamp.

4.6 Backup & Disaster Recovery

- **Database Backups:** Automated full backups of the primary relational database must occur **every 24 hours**, with Point-in-Time Recovery (PITR) enabled for up to 7 days to protect financial (Debt engine) and medical records.
- **Storage Redundancy:** Uploaded medical documents (Vault) must be stored in a highly durable object storage system (e.g., AWS S3 standard) with a 99.9999999% durability target.

Adding **Redis** alongside a Spring Boot monolith is a massive architectural win. Even though you aren't using microservices, using Redis to handle the "heavy lifting" of high-frequency GPS pings prevents your PostgreSQL database from being crushed under the load of real-time tracking.

Here is the finalized and updated **Step 5: Technical Architecture** incorporating Redis.

5 Technical Architecture

This section outlines the concrete technology stack and architectural patterns used to deliver the OmniCare MVP. To ensure rapid development and high transactional integrity for Phase 1, the backend will utilize a robust monolithic architecture augmented by an in-memory datastore for real-time performance.

5.1 Technology Stack

Frontend (Client Applications)

- **Mobile Application (Patient & Doctor Apps):** Flutter (Dart). Chosen for compiling to native iOS and Android from a single codebase, ensuring high performance for map rendering and GPS tracking.
- **Web Application (Admin & Dashboards):** React 18+ with TypeScript and Tailwind CSS v4. Designed strictly for desktop/browser management of schedules, profiles, and financial revenues.

Backend & Infrastructure

- **Core Framework:** Spring Boot (Java). A monolithic architecture for the MVP to centralize business logic, simplify deployment, and guarantee ACID-compliant financial transactions.
- **API Architecture:** RESTful APIs for standard CRUD operations.
- **Real-Time Communication:** Spring WebSockets (STOMP protocol) to handle live GPS coordinate streaming.

Database, Caching & Storage

- **Primary Database:** PostgreSQL. Used exclusively for permanent records (Users, Medical Passports, Financial Debt ledgers, Consultation History).
- **Geospatial Queries:** Hibernate Spatial integrated with PostGIS (via PostgreSQL) to handle radius searches and matching algorithms natively.
- **High-Frequency/Real-Time Store: Redis.** Acts as an in-memory datastore and Pub/Sub broker. Used to store volatile data like the "Live GPS Coordinates" of moving doctors and their "Online/Offline" availability status.
- **Object Storage:** AWS S3 (or compatible local alternative) for securely storing prescription photos and medical documents.

Third-Party Integrations

- **Mapping & Routing:** Google Maps Platform (Maps SDK for Flutter, Places API, Directions API).
- **Authentication & SMS:** Spring Security integrated with a local Tunisian SMS Gateway (e.g., Orange/Ooredoo or Twilio).
- **Social Login:** Google Identity Services for the initial OAuth flow.

5.2 Architecture Design & Data Flow

The system operates on a unified monolithic backend, intelligently separating volatile real-time data (Redis) from permanent transactional data (PostgreSQL):

1. The Static Flow (REST API):

- a. The Flutter app requests a Medical Passport.
- b. The request hits the Spring Boot REST Controller.
- c. Spring Security validates the JWT token.
- d. Spring Data JPA fetches the record from PostgreSQL and returns the JSON payload.

2. The Real-Time Flow (Redis + WebSockets):

- a. The Doctor's Flutter app uses the device's native GPS to emit a coordinate (Lat/Lng) every 3-5 seconds.
- b. This payload is sent via WebSocket to the Spring Boot server.
- c. **The Redis Optimization:** Spring Boot immediately updates the doctor's current location in **Redis** (which handles thousands of writes per second effortlessly) instead of writing to the PostgreSQL disk.
- d. Spring Boot then broadcasts this coordinate to the specific WebSocket topic subscribed to by the Patient's Flutter app to animate the car marker on Google Maps.

3. The Transactional Flow (Closing & Debt):

- a. Doctor submits the "Fast-Report Closing" form via the Flutter app.
- b. The prescription photo is uploaded to the S3 bucket.
- c. The Spring Boot service initiates an @Transactional database operation.
- d. It saves the medical report, links the S3 photo URL, and updates the Doctor's Platform Debt balance in PostgreSQL simultaneously. If any step fails, the entire transaction rolls back.

5.3 Deployment Strategy

- **Backend Hosting:** The Spring Boot application will be containerized using Docker and deployed as a single service on a cloud provider (e.g., Railway, AWS Elastic Beanstalk, or a VPS).
- **Data Layer:** A managed PostgreSQL instance and a managed Redis instance.
- **Web Hosting:** The React dashboard will be built into static files and served via a CDN (e.g., Vercel, Netlify).

Here is **Step 6: UX/UI Requirements** for your *Cahier de Charge*.

Because this application handles stressful emergency situations for patients and fast-paced environments for doctors, the design cannot just be "pretty"—it must be frictionless. This section locks in those critical usability constraints.

UX/UI Requirements

This section defines the user experience principles, accessibility standards, and visual guidelines that the OmniCare platform must strictly adhere to across both mobile and web interfaces.

6.1 Design System & Branding

- **Strict Color Palette:** The UI must exclusively use the color palette provided by the client to maintain the "Medical Trust" aesthetic.
 - Primary: Seafoam Green (#1FBF9A) - Used for primary actions (e.g., "Request Doctor", "Confirm").
 - Secondary: Deep Teal (#0F6F73) - Used for active states and typography emphasis.
 - Background: Light Grey/Off-White (#F4F5F7) - Used to reduce eye strain.
- **Typography:** Plus Jakarta Sans. Chosen for its high legibility on mobile screens, particularly for reading numeric vital signs and medical terminology.
- **Iconography:** Lucide React (for Web) and native Material/Cupertino equivalents in Flutter, utilizing clear, universally understood medical and navigational icons.

6.2 Platform-Specific Interfaces

- **Patient & Doctor Mobile Apps (Flutter):**
 - **Mobile-First Urgency:** The UI must be optimized for one-handed use. Critical action buttons (like the Emergency Request slider) must be placed in the lower third of the screen ("Thumb Zone").
 - **Safe Area Handling:** The design must automatically respect iOS and Android safe areas (notches, dynamic islands, and home indicator bars) to prevent UI overlap.
- **Admin & Doctor Dashboards (React Web):**
 - **Desktop-Optimized:** Built specifically for tablet and desktop viewports (min-width: 768px). Data tables (Consultation History, Revenue, Patient Records) must use sticky headers and horizontal scrolling where necessary.

6.3 The Doctor UX (Critical Flow Constraint)

- **High-Speed Closing Flow:** The interaction design for the doctor's consultation closing flow is critical and must prioritize speed. It directly affects the doctor's UX both in the field and later in their dashboard.
 - **Zero-Typing Policy:** Input must rely on large touch targets (Chips for Diagnosis, Checkboxes for Treatments).
 - **Visual Confirmation:** The camera capture for the prescription must provide immediate, high-contrast visual feedback that the document was successfully attached to the patient's record.

6.4 Accessibility Standards (WCAG 2.1 AA)

- **High Contrast:** Text and interactive elements must maintain a minimum contrast ratio of 4.5:1 against their backgrounds. This is vital for users making requests in poorly lit environments or doctors working night shifts.
- **Error Prevention:** Destructive actions (e.g., "Cancel Request", "Delete Family Member") must require a two-step confirmation (e.g., a modal or swipe-to-confirm interaction) to prevent accidental triggers during high-stress moments.
- **Language Support:** The UI must support seamless switching between French (standard for Tunisian medical documentation) and Arabic, accommodating right-to-left (RTL) layout rendering where necessary.

6.5 Mockups & Wireframes Reference

- All frontend development must strictly match the approved Figma prototype layouts. Any deviation in spacing, color, or flow must be approved by the product lead.

This is a smart, lean approach. Relying strictly on the 15% transaction fee keeps your go-to-market strategy simple and avoids overcomplicating the MVP, while waiting for official regulatory approval before touching digital payments protects you legally.

Here is the corrected **Step 7: Monetization & Payment Model**.

7 Monetization & Payment Model

This section outlines OmniCare's sole revenue stream and the phased roadmap for handling financial transactions, strictly aligned with current technical scope and future regulatory milestones.

7.1 Phase 1 (MVP) Revenue Model: The "Platform Debt" System

For the initial launch, the platform operates without a live digital payment gateway. The platform's **only source of income** is a transaction fee. There are no monthly SaaS subscriptions or "Pro" tiers for doctors.

- **Patient Cost:** 100% Free to use the app. Patients only pay the cost of the medical consultation directly to the doctor.
- **Platform Revenue (The 15% Rule):** The platform charges a flat **15% commission** on the total fee of every successfully completed emergency home visit.

The Financial Logic (Cash & Debt Reconciliation):

1. **Transaction:** The patient pays the doctor entirely in **Cash (Espèces)** at the physical location.
2. **Calculation:** When the doctor submits the "Fast-Report Closing" form, they input the final collected amount.
3. **Debt Ledger:** The Spring Boot backend automatically calculates 15% of that amount and adds it to the doctor's Platform Debt balance in the PostgreSQL database.
4. **Enforcement (The 100 TND Limit):** If a doctor's cumulative Platform Debt exceeds **100 TND**, their account is instantly suspended. They will be locked out of the dispatch system and cannot accept new requests until they settle their debt with the OmniCare administration manually (e.g., via direct bank transfer or cash deposit).

7.2 Phase 2: Regulated Digital Payments (Post-Approval)

Once official government and regulatory approvals for digital health transactions are secured, OmniCare will integrate **ClicToPay** as the official payment gateway.

- **Doctor Debt Settlement:** Doctors will be able to clear their 15% Platform Debt directly within their application using their credit/debit cards via the ClicToPay integration, automating the reconciliation process.
- **Patient Digital Checkout:** Patients will have the option to pay for the entire consultation upfront via ClicToPay within the OmniCare app.
- **Automated Split:** For digital payments, the backend will automatically split the funds—routing 85% to the doctor's connected account and retaining the 15% platform commission, completely eliminating the manual debt accumulation for those specific transactions.

7.3 Refund & Cancellation Policy

- **Patient Cancellation:** A patient can cancel a request free of charge *before* the doctor clicks "Accept." Frequent cancellations after a doctor is dispatched will result in an automated account ban.
- **Disputes (Phase 1):** Since MVP transactions are strictly Cash-on-Delivery, OmniCare is not liable for financial refunds. The platform acts solely as the logistical connector. Fee disputes must be resolved directly between the patient and the provider at the time of the visit.

This is a fantastic extraction from Windsurf. It gives us the exact anatomy of your front-end. By mapping this directly into our PostgreSQL schema, we ensure your front-end React/Flutter components will perfectly consume the JSON APIs your Spring Boot backend generates.

I have taken the frontend state, added the **Authentication layer**, integrated the **Dynamic Doctor Fees (Bidding)**, and added the **Admin Dashboard** entities.

Here is your finalized, production-ready **Step 8: Data Model**.

Data Model

This section defines the core database entities, their attributes, and the relational mapping for the Spring Boot / PostgreSQL architecture. It bridges the current frontend UI state with a secure, ACID-compliant backend.

8.1 Core & Identity Entities

1. users (Authentication & Authorization)

- *Missing from MVP demo, required for Production.*
- id (UUID, Primary Key)
- phone_number (String, Unique) - *Primary login*
- email (String, Optional) - *For Google OAuth*
- password_hash (String)
- role (Enum: PATIENT, DOCTOR, ADMIN)
- is_active (Boolean) - *Controlled by Admin*
- created_at / last_login (Timestamp)

2. doctor_profiles (Provider Side)

- user_id (UUID, Primary Key, Foreign Key -> users.id)
- first_name / last_name (String)
- specialty (String)
- cnom_id (String, Unique) - *Medical License ID*
- avatar_url (String)
- bio / experience (String)
- languages (JSONB) - *Array of strings*
- verification_status (Enum: PENDING, VERIFIED, REJECTED)
- rating_average (Decimal) / review_count (Integer)
- platform_debt (Decimal) - *Tracks the 15% owed*
- is_online (Boolean) - *For real-time map presence*

8.2 Medical Data Entities (Carnet de Santé)

3. family_members (Patient Side)

- id (UUID, Primary Key)
- owner_user_id (UUID, Foreign Key -> users.id) - *The "Chef de Famille"*
- relationship (Enum: SELF, BABA, MAMAN, ENFANT, AUTRE)
- full_name (String)
- age (Integer)
- qr_code_hash (String, Unique)

4. medical_passports (Health Record)

- family_member_id (UUID, Primary Key, Foreign Key -> family_members.id)
- blood_type (Enum: A+, O-, etc.)
- chronic_conditions (JSONB) - *Array (e.g., ["Diabète Type 2", "Asthme"])*
- current_medications (JSONB) - *Array of objects: [{name, dosage, frequency}]*
- critical_alert (String, Optional)

5. medical_documents (Secure Vault)

- id (UUID, Primary Key)
- family_member_id (UUID, Foreign Key -> family_members.id)
- document_type (Enum: BLOOD_TEST, XRAY, SCAN, PRESCRIPTION, REPORT)
- file_url (String) - *Secure S3 Bucket URL*
- document_date (Date)

8.3 The Emergency Matching Engine (Bidding & Dynamic Fees)

6. emergency_requests (The Dispatch)

- id (UUID, Primary Key)
- patient_user_id (UUID, Foreign Key -> users.id)
- family_member_id (UUID, Foreign Key -> family_members.id)
- symptoms_data (JSONB) - *Stores bodyParts array, painLevel (1-10), and description*
- emergency_location (Geometry/Point - PostGIS) - *Exact GPS coordinates*
- status (Enum: BROADCASTING, DOCTOR_SELECTED, CANCELLED)

7. doctor_bids (Dynamic Fee Mechanism)

- *This powers the UI where the patient sees 3 doctors with different prices (100 TND, 150 TND, 80 TND).*
- id (UUID, Primary Key)
- request_id (UUID, Foreign Key -> emergency_requests.id)
- doctor_user_id (UUID, Foreign Key -> users.id)
- proposed_fee (Decimal) - *The price the doctor inputs to accept the job*
- estimated_eta (Integer) - *In minutes*
- status (Enum: PENDING, ACCEPTED_BY_PATIENT, REJECTED)

8.4 Transactions & Admin Tracking

8. consultations (The Final Record)

- id (UUID, Primary Key)
- bid_id (UUID, Foreign Key -> doctor_bids.id) - *Links to the agreed price and ETA*
- status (Enum: EN_ROUTE, ARRIVED, COMPLETED, CANCELLED)
- final_fee_collected (Decimal) - *Confirmed by Doctor at closing*
- platform_fee_generated (Decimal) - *15% calculated automatically*
- closing_report (JSONB) - *Stores diagnosis, treatments_performed*
- prescription_file_url (String) - *Auto-saves to medical_documents*
- payment_method (Enum: CASH, CARD)

9. platform_transactions (Revenue Ledger for Admin)

- id (UUID, Primary Key)
- consultation_id (UUID, Foreign Key -> consultations.id)
- doctor_user_id (UUID)
- gross_amount (Decimal)
- platform_fee (Decimal)
- net_doctor_earnings (Decimal)
- transaction_date (Timestamp)

10. admin_audit_logs (Security & Tracking)

- id (UUID, Primary Key)
- admin_user_id (UUID, Foreign Key -> users.id)
- action_type (String) - *e.g., "VERIFY_DOCTOR", "SUSPEND_ACCOUNT"*
- target_entity_id (UUID)

- timestamp (Timestamp)

8.5 Relational Mapping Summary

- **Patient Flow:** 1 User (Chef de Famille) -> Many family_members -> 1 medical_passport per member.
- **Dispatch Flow:** 1 emergency_request -> receives Many doctor_bids -> Patient selects 1 Bid -> generates 1 consultation.
- **Revenue Flow:** 1 consultation -> generates 1 platform_transaction -> updates platform_debt in doctor_profiles.

Security & Compliance

This section defines the security protocols, encryption standards, and legal compliance measures required to protect user data and ensure the platform operates within the bounds of Tunisian law.

9.1 Regulatory Compliance (Tunisian Law)

The platform must strictly comply with the **Loi organique n° 2004-63 du 27 juillet 2004** relative to the protection of personal data, overseen by the **INPDP** (Instance Nationale de Protection des Données Personnelles).

- **Health Data Processing (Articles 62 & 63):** * Explicit, documented consent must be obtained from the patient (or their legal guardian/tuteur) before collecting their "Carnet de Santé" data.
 - Medical Passports can *only* be accessed by verified medical professionals bound by professional secrecy.
- **User Rights:** The system must provide mechanisms for patients to exercise their legal rights, including:
 - **Droit d'accès & d'oubli:** Patients can view, export, or permanently delete their Medical Passports and uploaded Vault documents at any time.

- **Data Localization:** If AWS S3 or external cloud servers are used to host the database or document vault outside of Tunisia, OmniCare must file a formal declaration and seek authorization from the INPDP prior to deployment.

9.2 Data Security & Encryption

- **Data in Transit:** All API communication between the Flutter mobile apps, the React web dashboard, and the Spring Boot backend must be encrypted using **TLS 1.2 or higher** (HTTPS/WSS).
- **Data at Rest:** * The PostgreSQL database volume must be encrypted.
 - Highly sensitive columns (such as password_hash, blood_type, chronic_conditions, and current_medications) must use application-level encryption (AES-256) before being written to the database.
 - Medical documents in the S3 Vault must be encrypted at rest, and accessed only via short-lived, pre-signed URLs.

9.3 Role-Based Access Control (RBAC)

Strict isolation must be enforced by Spring Security via JWT (JSON Web Tokens).

- **Patient Role:** Can only access and modify their own family_members and associated medical data. Cannot view doctor locations unless an emergency request is active.
- **Doctor Role:** * Must be verified by an Admin before gaining access to the dispatch map.
 - **Critical Rule:** A doctor is granted read-only access to a patient's Medical Passport *only* upon accepting their emergency request. This access is instantly revoked by the system the moment the consultation status changes to COMPLETED or CANCELLED.
- **Admin Role:** Full read/write access to user verification statuses and financial ledgers, but strictly restricted from viewing individual patient Medical Passports or Vault documents.

9.4 Audit Logging & Monitoring

To prevent abuse and ensure accountability, the Spring Boot backend must maintain an immutable admin_audit_logs table.

- The system must log every instance where a Doctor accesses a Patient's Medical Passport (Timestamp, Doctor ID, Patient ID).
- The system must log all Admin actions, specifically the modification of a Doctor's verification_status or manual adjustments to the platform_debt.

9.5 Application & Network Security

- **Authentication Rate Limiting:** The SMS OTP endpoint must be rate-limited to prevent brute-force attacks (e.g., maximum 3 attempts per 5 minutes, followed by a 15-minute logout).
- **SQL Injection & XSS Prevention:** Spring Data JPA automatically sanitizes inputs against SQL injection. The React frontend must strictly sanitize all user inputs (like symptom descriptions) to prevent Cross-Site Scripting.
- **Session Management:** JWT tokens must have a short lifespan (e.g., 1 hour for access tokens), utilizing a secure HttpOnly refresh token mechanism to maintain sessions without exposing credentials to local storage vulnerabilities.

Testing & Quality Assurance (QA)

This section defines the rigorous testing protocols required to ensure the OmniCare platform is reliable, secure, and performant under the unpredictable conditions of mobile networks and high-stress medical dispatching.

10.1 QA Strategy for Healthcare (Compliance First)

- **Data Anonymization:** Under no circumstances will real patient "Carnet de Santé" (Medical Passport) data be used in Staging or QA environments. All test databases must use generated, dummy data to comply with INPDP regulations and ensure patient privacy.
- **Role-Based Access Control (RBAC) Matrix:** QA must rigorously test the boundaries of the security system. Specifically, automated tests must verify that a Doctor's read-only access to a medical_passports record instantly expires the exact second a consultation status shifts to COMPLETED or CANCELLED.

10.2 Mobile App Testing (Flutter)

- **GPS Mocking & Live Tracking:** To test the core matching engine without physically driving around Tunis, QA will use location-spoofing tools (like Android's Mock Location) and Flutter widget testing to mock the geolocator and Maps_flutter streams. The goal is to verify that the car icon moves smoothly on the patient's screen without visual jitter.
- **Background Execution & Device Sleep:** * *Critical Test:* The doctor's app must be tested for "Foreground Service" and Wake Lock survival. If the doctor locks their screen while En_Route, the app must continue broadcasting GPS coordinates to the Redis backend.
- **Offline Tolerance & Network Drops:** QA must simulate a 4G network drop exactly as the doctor hits "Submit" on the Fast-Report Closing form. The Flutter app must cache the payload locally and auto-sync when the connection returns, ensuring the prescription photo and the 15% debt calculation are flawlessly transmitted without forcing the doctor to re-type the diagnosis.

10.3 Backend & Infrastructure Testing (Spring Boot & Redis)

- **WebSocket & Redis Load Testing:** The most resource-intensive part of the architecture is the real-time tracking. QA will use load-testing tools (like Apache JMeter or Gatling) to simulate 500+ doctors simultaneously pinging their GPS coordinates every 3 seconds into the Spring Boot STOMP endpoints and Redis Pub/Sub channels to ensure the server maintains sub-200ms latency without crashing.
- **Integration Testing with Testcontainers:** To ensure the PostGIS geographic matching (finding doctors within a specific radius) and the financial ledgers are accurate, the CI/CD pipeline will utilize **Testcontainers**. This approach spins up ephemeral, isolated PostgreSQL and Redis Docker instances to run automated tests against the actual database logic before any code is merged to production.

10.4 Security & Penetration Testing

- **Session & Token Lifecycle:** Testing the 5-minute expiry of the SMS OTPs and verifying that compromised JWT access tokens cannot be reused.

- **S3 Vault Security Validation:** Verifying that prescription images and medical documents stored in the AWS S3 buckets cannot be accessed via direct public URLs, but only through temporary, pre-signed URLs requested by an actively authenticated session.

1 Deployment Strategy & Project Roadmap

11.1 Infrastructure & Hosting Environments

The architecture is designed to be lean for Phase 1 while maintaining enterprise-level stability.

- **Backend (Spring Boot & PostgreSQL):** The monolithic backend and PostgreSQL database will be deployed on a managed PaaS (Platform as a Service) like Railway or Render. This abstracts away server management, allowing for instant Dockerized deployments.
- **Real-Time Data (Redis):** A managed Redis instance will be provisioned alongside the backend to handle the high-frequency GPS pinging.
- **Web Dashboard (React):** The Admin and Doctor web portals will be built as static assets and hosted globally on a CDN like Vercel or Netlify for sub-second load times.
- **Mobile Apps (Flutter):**
 - **Android:** Deployed via Google Play Console (Internal Testing -> Open Beta -> Production).
 - **iOS:** Distributed via Apple TestFlight for initial beta testers, requiring a strict App Store review for the final production release.

11.2 CI/CD Pipeline (Continuous Integration / Continuous Deployment)

To ensure code quality and prevent manual bottlenecks, all deployments will be automated using **GitHub Actions**.

- **Pull Request (PR) Checks:** Every push to the main branch automatically triggers a build to verify the Spring Boot codebase compiles, runs unit tests, and executes Flutter analyze and test commands.
- **Automated Dockerization:** The Spring Boot backend uses a multi-stage Dockerfile leveraging Eclipse Temurin (Alpine) to keep the image size under 150MB. This image is automatically built and pushed to the container registry on merge.
- **Robust Observability:** The production environment will stream structured application logs and metrics directly to a centralized logging tool (e.g., Datadog or ELK stack). Having this deeply integrated from day one ensures that tracing down complex user issues or failed transactions feels like a standard level 2 technical support workflow, rather than a guessing game.

11.3 Local Development Setup

Since the primary development machine is a MacBook Air (rouge), the local environment relies on macOS-native package managers for absolute consistency.

- **System Requirements:** Homebrew is strictly used to manage dependencies.
 - brew install openjdk@17 (Spring Boot core)
 - brew install maven
 - brew install postgresql postgis
 - brew install redis
- **Mobile Simulation:** The iOS simulator (via Xcode) and Android Emulator (via Android Studio) will be used to test the Flutter apps side-by-side, specifically simulating GPS location spoofing for the tracking engine.

11.4 Phase 1 MVP Roadmap (4-Month Timeline)

This is a realistic schedule designed to accommodate an intensive, night-shift study schedule without burning out development resources.

- **Month 1: Foundation & Architecture**
 - Configure PostgreSQL, PostGIS, and Redis schemas.
 - Setup Spring Boot Spring Security (JWT) and SMS OTP integration.
 - Develop Flutter UI Shell and React Admin UI Shell.
- **Month 2: The Core Loop (CRUD & Profiles)**
 - Implement the Medical Passport and Document Vault (S3 upload).

- Build the Doctor verification flow in the Admin dashboard.
- *Milestone:* Patients can create full family profiles and upload documents.
- **Month 3: The Engine (Real-Time Dispatch)**
 - Implement the 7-Step Emergency Request wizard.
 - Build the STOMP WebSockets and Redis Pub/Sub logic for location tracking.
 - Implement the Google Maps UI in Flutter for live route rendering.
 - *Milestone:* A patient request successfully pings a mocked doctor on the map.
- **Month 4: Financials, QA & Launch**
 - Build the "Fast-Report Closing" flow and the 15% Platform Debt logic.
 - Conduct rigorous QA testing (Location spoofing, network drop simulation).
 - Submit Flutter apps to App Store and Google Play.
 - *Milestone:* Live Beta test with 5 trusted doctors in Grand Tunis.